

A data-driven matching algorithm for ride pooling problem

Ahmet Şahin^{a,*}, İsmail Sevim^b, Erinc Albey^a, Mehmet Güray Güler^b

^a Department of Industrial Engineering, Özyeğin University, Cekmekoy Campus, Nisantepe District, Orman Street, 34794, Istanbul, Turkey

^b Department of Industrial Engineering, Yıldız Technical University, Barbaros Bulvarı, 34349, Istanbul, Turkey

ARTICLE INFO

Keywords:

Machine learning
Ride matching
Rank aggregation
Binary programming
Nonlinear bilevel programming

ABSTRACT

This paper proposes a data-driven matching algorithm for the problem of ride pooling, which is a transportation mode enabling people to share a vehicle for a trip. The problem is considered as a variant of matching problem, since it aims to find a matching between drivers and riders. Proposed algorithm is a machine learning algorithm based on rank aggregation idea, where every feature in a multi-feature dataset provides a ranking of candidate drivers and weight for each feature is learned from past data through an optimization model. Once weight learning and candidate ranking problems are considered simultaneously, resulting optimization model becomes a nonlinear bilevel optimization model, which is reformulated as a single level mixed-integer nonlinear optimization model. To demonstrate the performance of the proposed algorithm, a real-life dataset from a mobile application of a ride pooling start-up company is used and company's current approach is considered as benchmark. Results reveal that proposed algorithm correctly predicts the first choice of riders 17% to 28% better compared to the benchmark in different scenarios. Similarly, proposed algorithm offers recommendation lists in which the preferred driver is ranked 0.38 to 1.12 person closer (to the rider's actual choice) compared to the benchmark.

1. Introduction

In 2019, direct and indirect traffic congestion cost is estimated to reach more than \$88 billion in USA (Anon, 2020). This huge loss is attributed to several factors, such as cost of wasted fuel, lost productivity of individuals stuck in traffic and increased cost of transporting goods through congested areas (Anon, 2018a). Public transportation is one of the most effective means of reducing congestion caused by commuters, since it significantly reduces the number of private vehicles required to respond to transportation needs of individuals. However, for some commuters and for many individuals, public transportation is not an option. For such cases, one possible way to reduce traffic is to increase the utilization of private vehicles. Several reports indicate that, on the average, 1.5 people travel in a single vehicle (Anon, 2018b,c). Considering the available capacity of vehicles, merging transportation needs – through efficient matching strategies, that consider commuters and riders preferences – could decrease traffic congestion considerably.

Merging transportation needs of individuals is not a new problem, however, with the changing technology and user habits, its structure has completely changed. It is now possible, more than ever, to track availability of cars and transportation needs of individuals online. This transformation gives rise to new ecosystems, where individuals looking for a ride can be matched with others, who are willing to pool their

ride. These ecosystems, mostly mobile applications, allow users to engage in a ride pooling activity under certain terms and conditions. For such ecosystems, ride pooling is referred as a transportation mode, in which individuals share a vehicle for a trip (Agatz et al., 2012; Furuhashi et al., 2013). The typical structure of *ride pooling* as a way of transportation is as follows: On one side, there are drivers, who have predetermined routes and departure times. On the other side, there are riders, who are willing to travel from an origin to a destination and looking for a mean of transportation. In between, there is a matching agency. The agency's main objective is collecting relevant trip information, such as routes and departure time preferences of drivers and riders, and matching them considering the available information. Once, the agency matches a driver and a rider, it offers them the matching and expect them to accept the offer. This process is called **ride matching** and addressed in the literature (Agatz et al., 2012; Furuhashi et al., 2013; Hou et al., 2018). Note that matching a rider and a driver is more complex than assigning a driver to a rider in UBER-like setting (Cramer and Krueger, 2016), since in the ride matching problem the rider chooses a ride-pal rather than picking up a cab with an anonymous driver. Although ridepool/carpool has great potential for commuting, it is reported that the ratio of people that prefers these

* Corresponding author.

E-mail addresses: ahmet.sahin@ozu.edu.tr (A. Şahin), isevim@yildiz.edu.tr (İ. Sevim), erinc.albey@ozyegin.edu.tr (E. Albey), mgguler@yildiz.edu.tr (M.G. Güler).

<https://doi.org/10.1016/j.cor.2021.105666>

Received 15 March 2021; Received in revised form 10 October 2021; Accepted 2 December 2021

Available online 25 December 2021

0305-0548/© 2021 Published by Elsevier Ltd.

services reduces every year (Anon, 2018d; McKenzie, 2015). This shows that people (riders and drivers) cannot benefit from ride pooling well enough. Therefore, ride pooling agencies should dedicate their relevant resources to popularize their applications.

A ride pooling agency that aims to promote its application must provide flexible, convenient, and reliable services. Moreover, it must devise methods to motivate riders and drivers (Agatz et al., 2012). Flexibility is defined as the capability of a ride pooling agency to readjust the time schedules and itineraries with respect to unexpected changes. Convenience refers to the idea of incorporating drivers' and riders' preferences into ride matching processes and a reliable ride pooling agency is characterized by its ability to provide a service whenever it is requested. Matching rate, as a performance measure of reliability, is given as the ratio of accepted matching offers to all matching offers (Nourinejad and Roorda, 2016; Agatz et al., 2012) and it is concluded that increasing the matching rate is a crucial task for a ride pooling agency (Jacob and Roet-Green, 2018). To increase the matching rate, the agency ensures the usage of an effective ride matching procedure.

An UBER-like application offers crowd sourced transportation and matches available drivers with people, who need rides. In UBER, drivers are contractors of UBER, and similar to taxi drivers, they take people from point A to B for a fee. In ridepooling, however, drivers are neither employees nor contractors, and they have their own routes. Drivers share their ride only if a rider has a similar route. Moreover, satisfaction of a set of criteria is also needed for matching. Such decentralization complicates the matching in ridepooling, and results in algorithmic challenges. From a methodological perspective of the peer-to-peer ride matching algorithms, the literature is mainly divided into three categories: (i) one-to-one ride matching, (ii) one-to-many ride matching, and (iii) multihop ride matching (Tafreshian et al., 2020). In one-to-one ride matching, a driver/rider pair shares a single ride without any transfers, and in one-to-many ride matching, a driver shares her ride with multiple riders. The last category is actually an umbrella term includes both many-to-one and many-to-many ride matching. Intermediate transfers between drivers is allowed (e.g., a rider can have multiple cars in her route) in multihop ride matching, and this property distinguishes it from the first two categories. Various modelling approaches, and exact/heuristic solution methods are proposed for all ride matching categories. In the following paragraph, we briefly review the one-to-one ride matching literature, since the ride matching algorithm in this study belongs to this line of research.

In Ta et al. (2017), authors propose a ride matching algorithm based on the requirement that the shared route percentage of driver/rider pair exceeds a predefined amount. As the first step of the ride matching procedure, possible driver/rider pairs do not satisfy the aforementioned requirement are pruned. Then, a weighted bigraph matching problem is optimally solved to obtain driver/rider pairs as one-to-one ride matches. A more elaborated model incorporating time-uncertainty into ride matching is proposed in Long et al. (2018). The ride matching procedure starts with spatiotemporal pruning for all drivers and riders to define the feasible matches. Then a static ride matching problem with stochastic travel times is solved by Monte Carlo simulation for all drivers and riders such that the total travel costs are minimized, and the number of matches is maximized. Authors in Thangaraj et al. (2017) propose a ride matching procedure consists of three core components. In the preprocessing component, the targeted region is divided into clusters. Following the deployment of the system, when a ride matching request is provoked, all possible matches are pruned by geographical (e.g., drivers in remote clusters are not preferred) and temporal considerations. Then, the remaining set of matches are submitted to the rider in form of a list. The last component, namely in-memory indexing, deals with storing the data related to matches. However, authors do not give any details about how to exploit this stored data. In a more recent study, Kumar and Khani (2021), a ride matching algorithm integrating ride pooling and schedule-based transit system is

proposed. The algorithm is a composition of the concept of space-time prism and a shortest path labelling algorithm. Space-time prism deals with spatiotemporal pruning as a preprocess to ride matching. Then, a procedure based on shortest path labelling and Bellman's principle of optimality is provoked to obtain a set of possible matches and this set of matches is submitted to the rider in the form of a list. For an up-to-date comprehensive review of matching algorithms, readers are referred to Tafreshian et al. (2020).

In Fan et al. (2018) and Bozdog et al. (2018), it is discussed that the matching can be done through geographical and temporal pruning. Since, the agency have the information of drivers' and riders' departure time and locations, it can match drivers and riders that are close to each other in terms of location and departure time by using rule based methods. On the other hand, it can be argued that mathematical model based approaches find more acceptable matching offers. In Hou et al. (2018), Tian et al. (2013), Herbawi and Weber (2012) and Li et al. (2014), authors model the ride matching problem as variants of a dial-a-ride problem, a well-studied problem in the operations research literature (Riedler and Raidl, 2018; Masmoudi et al., 2017; Cordeau and Laporte, 2007), which are also based on geographical and temporal pruning. Siddiqi and Buliung (2013) allows riders to choose any preferred driver, instead of pruning and matching the participants. Authors discussed that such a method is inefficient, because the riders tend to call their friends. A different approach is given in Sánchez et al. (2016), where the authors proposed the idea of tag-based matching. In tag-based matching, a driver put tags on her trip and that trip is offered only to the riders who are interested in one or more of these tags. However, the performance of the approach is not presented in the study.

In this study, we propose a ride matching algorithm by respecting to a set of principles of decision theory literature (Saaty, 1988): (i) agents make their decisions based on a set of criteria, and (ii) weights of each criterion may differ from each other. Applying these principles to ride pooling, we claim that a rider bases her/his decision on a set of criteria, and each criterion has its own weight. For each criterion, a rider has a ranking of matching offers and tends to accept an offer with higher ranks. However, since, the rankings of offers in each criterion may differ, a rider has to combine them into a consensus ranking. This process is known as **rank aggregation** in the literature (Aledo et al., 2017; Badal and Das, 2018). The rank aggregation problem focuses on combining multiple rankings into a single ranking. Several studies are presented in different areas such as ranking results of web engines (Dwork et al., 2001), aggregating user ratings (Baskin and Krishnamurthi, 2009), and the determination of the winner in sports competitions (Betzler et al., 2014). In literature, the algorithms are proposed for the rank aggregation problem in three main categories: positional methods, comparison sort methods, and local search methods. For instance, Borda's method (Coppersmith et al., 2010) is a positional method that finds a ranking that is minimum of the distances between each items' positions and their mean positions in the input rankings. Comparison sort algorithms compare each items' positions in rankings then sort the elements (Dwork et al., 2001; Ailon et al., 2008). Local search methods start with an initial ranking and try to improve a fitness function by moving items in each position (Schalekamp and Zuylen, 2009). In addition to these three main categories, a binary programming (BP) model is also proposed for the rank aggregation problem (Conitzer et al., 2006).

In ride pooling context, rank aggregation can be illustrated as follows. Assume that a rider has to decide one out of three drivers d_1 , d_2 , and d_3 to ride with, and she/he placed her/his request at 1:10pm. Rider's route matches 100% of d_1 's route, 80% of d_2 's route, and 60% of d_3 's route. Hence, a ranking based on route match is given as $d_1 > d_2 > d_3$. Let starting points of all drivers be same with the rider's starting point. Since, all are the same, this information returns a ranking where all drivers share the same rank: $d_1 = d_2 = d_3$. Assume that the departure times are 01:15 PM, 01:25 PM, and 01:25 PM respectively for d_1 , d_2 ,

and d_3 . Then, a ranking based on departure times is given as $d_1 > d_2 = d_3$. If, route match, starting point, and departure time are the three criteria for ride matching, then a rank aggregation can be provoked to obtain the consensus ranking $d_1 > d_2 > d_3$. Hence, the rider matches with d_1 . Now, consider the following example: There are two criteria and the rankings are $d_1 > d_2 > d_3$, and $d_2 > d_1 > d_3$. In the consensus ranking, d_1 and d_2 are in tie and this tie must be broken arbitrarily. It is possible to resolve this arbitrariness issue by assigning different weights to different rankings. If the weight of the first ranking is higher than the second one's, then the consensus ranking is $d_1 > d_2 > d_3$. Such a rank aggregation is called weighted rank aggregation and addressed in the literature (Pihur et al., 2007). However, there is no work incorporating the weighted rank aggregation problem into ride matching procedures in the literature.

The contribution of this study is threefold. First, it introduces the weighted rank aggregation into ride matching literature of ride pooling agencies. To do so, we update a Binary Program (BP) from the rank aggregation literature (Conitzer et al., 2006) by incorporating weights into the program. Second, we propose a machine learning method to learn these weights such that the matching rate is maximized. This is ensured by using a Nonlinear Bilevel Problem (NLBP) model. Third, we proposed a ride matching framework, LBRanker, that contains two parts (online and offline) be used as a ride matching procedure. In the offline part of the procedure, the machine learning method learns the weights for each criterion. Once the weights are learned, they are transferred to the online part to provoke a rank aggregation to obtain a consensus ranking whenever a trip request is placed. A static approach is considered here in the sense that when a trip request is placed, we assume that the list of available drivers is fixed. Dynamic version of the problem is out of scope of this study, since dynamicity is an operational issue rather than an algorithmic one. Online part of the procedure is periodically improved by relearning the weights by provoking the offline part with additional data collected during the execution of the online part.

We develop our algorithm considering the needs of Lojika, an Istanbul based start-up company. Lojika developed its ride pooling application, TAG, with funding received from the national science foundation of Turkey and European Union. TAG reached highest number of active users in city of Istanbul. Istanbul, with a population of more than 15 million, has a huge traffic congestion problem, such that it was a runner-up city in the traffic index announced by TomTom in 2013 and stays in top 10 since then Anon (2018e). TAG has a rule based ride matching algorithm, details of which will be provided in Section 2. Taking the existing matching algorithm in TAG, we develop our ride matching algorithm and test the performance of our method on the historical data from TAG. Results show that our algorithm performs significantly better than the current ride matching algorithm used in TAG, such that it correctly predicts the first choice of riders 17% to 28% better compared to the benchmark. Moreover, proposed approach offers recommendation lists in which the preferred driver is ranked 0.38 to 1.12 person closer (to the rider's actual choice) compared to the benchmark.

The rest of the paper is organized as follows. Details of the current ride matching approach used in the ride pooling company is presented in Section 2. We discuss the idea of using machine learning and optimization in ride matching problem and present our approach in Section 4, which is followed by numerical experiments and result discussions in Section 5. Last section, Section 6, presents the concluding remarks.

2. Current ride matching procedure: The benchmark

The proposed optimization based approach, LBRanker, is developed to replace the current rule-based approach of the company, which is also used as a benchmark during the numerical experiments in

Section 5. Therefore, for the sake of following a top-down approach, we first present the current ride matching procedure used in TAG.

The procedure of current approach in TAG follows a decentralized, peer-to-peer approach, i.e., it does not consider global matching of all driver/rider pairs. The procedure is provoked once a rider provides trip information (*origin-destination* pair), and preferred *departure time*. In this sense, it works in an online fashion. Then, the candidate driver list is prepared by selecting a ranked-subset from the set of all available drivers. The procedure prunes some of the available drivers based on a set of rules, and ranks the remaining ones to prepare the candidate driver list. In particular, it applies a three steps approach: (i) spatiotemporal pruning, (ii) criteria-based pruning, and (iii) ranking the drivers via *sorting*. In spatiotemporal pruning (Sánchez et al., 2016), the drivers out of the walking vicinity of the rider, and the drivers whose time windows do not intersect with the time window of the rider's trip are pruned. The procedure further prunes the remaining drivers with respect to a set of criteria in criteria-based pruning, e.g., percentage of route match, being co-members of an alumni association etc. The final step, ranking the drivers via *sorting*, ranks the drivers that survive through the pruning steps via a sorting algorithm, and eventually, it provides the candidate driver list. Once the list is ready, it is displayed to the rider and the rider is supposed to choose one or none of the drivers. If a driver is chosen, then the application informs the preferred driver and completes the ride matching with the acceptance of the driver. Otherwise, the procedure ends without a matching.

Flowchart of the ride matching procedure used in TAG is given in Fig. 1. The nomenclature used in pseudocode of the algorithm and the pseudocode are provided in Table 1 and in Algorithm 1, respectively. Let D be the set of all available drivers and p be the rider. Also, let o_p and a_p be the origin-destination points of rider p and r_d be the route of driver $d \in D$. v_1 and v_2 are the threshold parameters for the distance of origin-destination points. Please note that $\{e_p, l_p\}$ and $\{e_d, l_d\}$ are the pairs of earliest and latest departure times of rider $p \in P$ and driver $d \in D$, respectively. Also note that the operator $m(x, y)$ in Algorithm 1 calculates the minimum distance between point x and route y , and $sortandselect(x, y, z)$ sorts the members of set x by values in set y in direction determined by z where z is either increasing or decreasing and returns the top $min\{|x|, N_{max}\}$ members where N_{max} is the number of maximum candidate drivers shown to the rider on the mobile app.

Algorithm 1 Pseudocode of the ride matching procedure used in TAG.

```

1:  $CD \leftarrow \emptyset$ 
2:  $D_1 \leftarrow \emptyset$ 
3:  $D_2 \leftarrow \emptyset$ 
4: for  $d \in D$ :
5:   if  $m(o_p, r_d) \leq v_1$  and  $m(a_p, r_d) \leq v_2$ :
6:      $D_1 \leftarrow D_1 \cup d$ 
7: for  $d \in D_1$ :
8:   if  $(e_p, l_p) \cap (e_d, l_d) \neq \emptyset$ :
9:      $D_2 \leftarrow D_2 \cup d$ 
10: for  $d \in D_2$ :
11:   if  $\{v_k^d \geq \lambda_k \mid \forall \theta_k \in \Theta\}$ :
12:      $CD \leftarrow CD \cup d$ 
13:  $v_{Total} \leftarrow \emptyset$ 
14: for  $d \in CD$ :
15:    $v_{Total}^d \leftarrow 0$ 
16:   for  $c_k \in C$ :
17:      $v_{Total}^d \leftarrow v_{Total}^d + v_k^d$ 
18:    $v_{Total} \leftarrow v_{Total} \cup \{v_{Total}^d\}$ 
19:  $Ranking \leftarrow sortandselect(CD, v_{Total}, decreasing)$ 
20: return  $Ranking$ 

```

In this study, we adopt the framework of the existing ride matching procedure and replace the simple *sorting* operation of *Ranking* the

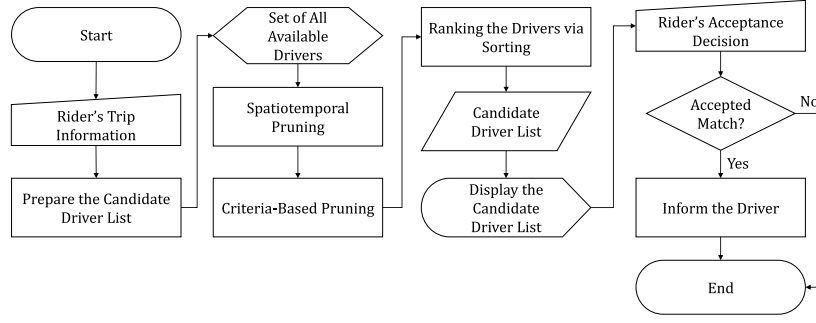


Fig. 1. Flowchart of TAG's ride matching procedure.

Table 1
Set definitions for current ride matching procedure.

Set	Members	Size	Description	Notes
D	$\{d_1, d_2, \dots, d_k, \dots, d_s\}$	s	Set of All Available Drivers	
R	$\{r_1, r_2, \dots, r_k, \dots, r_s\}$	s	Routes of All Available Drivers	
P	$\{p_1, p_2, \dots, p_k, \dots, p_q\}$	q	Set of All Riders	
O	$\{o_1, o_2, \dots, o_k, \dots, o_q\}$	q	Origins of All Riders	
A	$\{a_1, a_2, \dots, a_k, \dots, a_q\}$	q	Destinations of All Riders	
C	$\{c_1, c_2, \dots, c_k, \dots, c_n\}$	n	Criteria Set	
V^d	$\{v_1^d, v_2^d, \dots, v_k^d, \dots, v_n^d\}$	n	Criteria Scores for each $d \in D$	$v_k^d \in [v_k^{dl}, v_k^{du}]$ holds where $v_k^d, v_k^{dl}, v_k^{du} \in \mathbb{R}$
Θ	$\{\theta_1, \theta_2, \dots, \theta_k, \dots, \theta_m\}$	m	Subset of All Criteria for Candidate Selection	$\Theta \subset C, m \leq n$
Φ	$\{\phi_1, \phi_2, \dots, \phi_k, \dots, \phi_m\}$	m	Scores of Criteria Subset for Candidate Selection for driver $a \in A$	Θ and Φ are bijective, ϕ_k^a respects the boundary rules for its counterpart in H .
Λ	$\{\lambda_1, \lambda_2, \dots, \lambda_k, \dots, \lambda_m\}$	m	Acceptance Limits for Candidate Selection	Λ and Φ are bijective and λ_k respects the boundary rules for its counterpart in Φ .

Drivers via Sorting with a BP based weighted rank aggregation model. Furthermore, we propose an NLBP based machine learning method to learn the parameters of this model in an offline fashion.

3. Ranking the drivers: BP based weighted rank aggregation

In this study, we propose a ride matching algorithm consists of two parts: (i) online part works with a BP based weighted rank aggregation method to prepare the candidate driver list, and (ii) offline part uses a machine learning algorithm to determine the weights to be used in the weighted rank aggregation. In this section we discuss the details of the BP based weighted rank aggregation method. To do so, the mathematical model, and various approaches of calculating its parameters are discussed.

3.1. Mathematical model

Assume that spatiotemporal and criteria-based pruning are applied in response to a rider's request and a set of drivers, D , is obtained. The members of set D should be ranked before displaying the list to the rider as candidate drivers list. This step must also be done in an online manner. Following the context of rank aggregation, the BP model *Rank Aggregation with Single Trip (RAST)*, adapted from (Conitzer et al., 2006), can be used for ranking the drivers of a single trip.

RAST:

$$\min \sum_{i \in D} \sum_{j \in D, j > i} w_{ji} x_{ij} + w_{ij} x_{ji} \quad (1)$$

$$s.t. \quad x_{ij} + x_{ji} = 1 \quad \forall i, j \in D, i < j \quad (2)$$

$$x_{ij} + x_{jk} + x_{ki} \geq 1 \quad \forall i, j, k \in D, i \neq j \neq k \quad (3)$$

$$x_{ij} \in \{0, 1\} \quad \forall i, j \in D, i \neq j \quad (4)$$

In *RAST*, x_{ij} is equal to 1 if driver i precedes driver j , and 0 otherwise in the final ranking. Parameter w_{ij} represents the *pairwise superiority* of driver i to driver j , and satisfies the following conditions: (i) $w_{ij} \in [0, 1]$, and (ii) $w_{ij} = 1 - w_{ji}$. Basically, w_{ij} represents the fraction of criteria that driver i precedes driver j , and we addressed the issue of calculating w_{ij} parameters in detail in the following section. From now on, we will suppress the word *pairwise* and only use *superiority* while referring to parameter set w_{ij} . In *RAST*, Constraints (2) guarantees that either driver i or driver j precedes the other one. Constraints (3) satisfies the triangular inequality between drivers i , j , and k , e.g., if driver i precedes driver j and driver j precedes driver k , then driver i must also precede driver k . Constraints (4) are the binary constraints on x_{ij} variables. Please note that, *RAST* is for rank aggregation for a single trip of a rider. Later, we will extend the model to incorporate several trips of riders.

3.2. Calculating superiorities

Let Π be a set of criteria taken into account while ranking the drivers such that $\pi_1, \pi_2, \dots, \pi_k, \dots, \pi_n \in \Pi$ where $|\Pi| = n$. Here π_k is a permutation, i.e., it is an ordered set of drivers with respect to criteria k . Hence, drivers have precedence relations for each π_k . Precedence matrix W , adopted from (Aledo et al., 2013), is basically a matrix whose diagonal entries are zero and W_{ij} value represents the fraction of criteria that driver i precedes driver j , i.e., superiority value w_{ij} . By calculating the precedence matrix, one obtains w_{ij} values by using the following formula:

$$w_{ij} = \frac{1}{n} \sum_{k=1}^n \mathbf{1}(i <_k j), \quad (5)$$

where $i, j \in D$, $\mathbf{1}(\cdot)$ is an indicator function, and $\leq_k$ is an operator shows that left hand side driver precedes the other one in π_k . Each $w_{ij} \in W$ can be calculated using Eq. (5) for each $i, j \in D$ such that $i \neq j$, and transferred to *RAST* for rank aggregation. However, this calculation is only valid when π_k has full ranking, i.e., it is a permutation and for given two drivers there is exactly one winner. In driver ranking context, two drivers can have a tie in one or more criteria. For instance, rider's route may have an exact match with two distinct drivers' routes. Therefore, a modification of precedence matrix calculation is needed to calculate the superiorities. Let $s_{ij} \in S$ be the *auxiliary superiority point* of driver i over driver j for all $i, j \in D$ such that $i \neq j$. These points are unitless, not necessarily be in the closed set $[0, 1]$, and will be used for calculating the superiorities. An auxiliary superiority point is calculated by the following formula:

$$s_{ij} = \sum_{k=1}^n \mathbf{1}(i \leq_k j), \quad (6)$$

where $i, j \in D$, $\mathbf{1}(\cdot)$ is an indicator function, and $\leq_k$ is an operator shows that left hand side driver precedes the other one or they are in tie in π_k . One may calculate w_{ij} values by using the formula:

$$w_{ij} = \frac{s_{ij}}{s_{ij} + s_{ji}}. \quad (7)$$

Eq. (7) ensures that $w_{ij} = 1 - w_{ji}$ and $w_{ij} \in [0, 1]$ for all $i, j \in D$ such that $i \neq j$. Hence, these w_{ij} values can be transferred to *RAST* as the full-ranking case.

It is possible to resolve the problem of ties in individual rankings with calculating w_{ij} values via Eqs. (6)–(7). However, it is not possible to resolve the problem of breaking ties in a consensus ranking with such a calculation. Recall the rank aggregation example from Section 1, where d_1 and d_2 are in tie in the consensus ranking. Following the example, we denoted that this problem can be solved by using weighted rank aggregation by referring to the related literature. Here, we propose a formula that includes a set of weight coefficients ω_k for each criterion $\pi_k \in \Pi$ to calculate the auxiliary superiority points, and hence the superiorities w_{ij} . By such a calculation of w_{ij} values and feeding them to *RAST*, we introduce the model *Weighted Rank Aggregation with Single Trip (WRAST)*. The model is exactly the same with *RAST* with an exception in calculating the w_{ij} parameters. In *WRAST*, s_{ij} values are calculated by using the formula in Eq. (8), and superiorities w_{ij} is calculated by feeding these values to Eq. (7). The BP based weighted rank aggregation refers to the idea of obtaining a consensus ranking through the model *WRAST*.

$$s_{ij} = \sum_{k=1}^n \omega_k \times \mathbf{1}(i \leq_k j). \quad (8)$$

The weight coefficients, ω_k in Eq. (8), can be determined by using various methods. For example, as a naïve method, the values of all ω_k coefficients would be considered as equal, i.e., $\omega_1 = \omega_2 = \dots = \omega_k = \dots = \omega_n$. On the other hand, one may (i) refer to domain expert opinions to choose ω_k values or (ii) use a machine learning approach to learn them by referring to historical data. Naturally, we assume that there is at least one pair of distinct coefficients such that $\omega_l \neq \omega_m$ for both approaches.

4. Machine learning embedded ride matching: LBRanker

In this study, we propose a ride matching procedure, namely LBRanker, that consists of two parts: (i) Online part: Ranking the drivers with *WRAST* when a rider places a trip request, and (ii) Offline part: Periodically learning ω_k values with a machine learning algorithm. As previously described in Section 2, TAG's current ride matching procedure follows a three steps approach for preparing the candidate driver list: (i) spatiotemporal pruning, (ii) criteria-based pruning, and (iii) ranking the drivers via *sorting*. Ranking the members of a pruned list and sharing this ranked version with users is a

common approach in search-based applications. For instance, Google uses PageRank of the relevant web pages and rank them accordingly before displaying the search result (Brin and Page, 1998). Following this custom in the literature, for the online part of LBRanker, we adopt the first two steps of the current procedure, and replace the third step, i.e., ranking the drivers via *sorting*, with a BP based weighted rank aggregation model, i.e. *WRAST*, to increase the quality of the ranked lists. As the offline part of the proposed ride matching procedure, we introduce an NLBLP model to learn the ω_k values. Following sections are dedicated to the NLBLP model and its main building blocks. One may see the flowchart of LBRanker in Fig. 2.

4.1. The main building blocks

There are two multipliers in the summation of Eq. (8) for criterion π_k : (i) Criteria weight ω_k , and (ii) indicator function $\mathbf{1}(i \leq_k j)$. Output of the indicator function k for drivers i and j is directly calculated by using the ranking of criterion π_k . Hence, values of all indicator functions for criterion π_k are computed *online* once the ride matching procedure is provoked. On the other hand, calculating criteria weight ω_k is not a straightforward task and it needs closer inspection. In LBRanker, we propose that ω_k value can be calculated via a machine learning algorithm for each criterion π_k in an *offline* fashion. By offline, we mean that calculation of ω_k values are handled beforehand and they remain constant once ω_k values are calculated, unless a relearning is provoked. For this purpose, we introduce a novel NLBLP model as a machine learning algorithm. Before going into the details of the NLBLP model, main building blocks of the model are shared in this section. These blocks are the followings: (i) A BP for simultaneous rank aggregation, and (ii) a performance measure. In this section, we also give a basic information about bilevel programming, and a solution approach for the NLBLP model.

4.1.1. A BP for simultaneous rank aggregation

Ranking of drivers for a single rider's trip request is obtained through solving *WRAST*. However, we need a way to prepare rankings for multiple riders' trip requests simultaneously to be used in our machine learning algorithm. A binary program to serve such a purpose can be obtained with a slight modification to *WRAST*. Let T be the set of all riders' trip requests. Then, *Weighted Rank Aggregation with Multiple Trips (WRAMT)* is a BP to solve multiple weighted rank aggregation problems simultaneously.

WRAMT.

$$\min \sum_{t \in T} \sum_{i \in D_t} \sum_{j \in D_t | j > i} w_{ji}^t x_{ij}^t + w_{ij}^t x_{ji}^t \quad (9)$$

$$s.t. \quad x_{ij}^t + x_{ji}^t = 1 \quad \forall i, j \in D_t, i < j, \forall t \in T \quad (10)$$

$$x_{ij}^t + x_{jk}^t + x_{ki}^t \geq 1 \quad \forall i, j, k \in D_t, i \neq j \neq k, \forall t \in T \quad (11)$$

$$x_{ij}^t \in \{0, 1\} \quad \forall i, j \in D_t, i \neq j, \forall t \in T \quad (12)$$

In *WRAMT*, x_{ji}^t is equal to 1 if driver i precedes driver j in trip request t , and 0 otherwise in the final ranking. Parameter w_{ij}^t represents the superiority of driver i to driver j in trip request t , and calculated with Eqs. (7)–(8).

4.1.2. Performance measure

Assume that the candidate drivers are ranked by *WRAST* and displayed at the screen of the rider. If the rider accepts one of the drivers from the candidate driver list, the error of the given ranking can be calculated with the formula given below:

$$\phi = n - \sum_{j \in D | j \neq f} x_{fj}^* - 1 \quad (13)$$

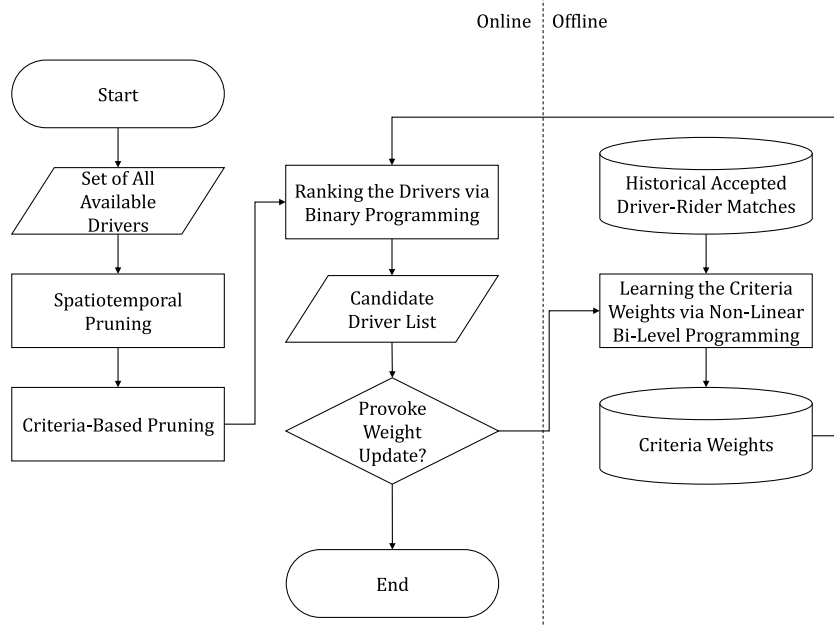


Fig. 2. The flowchart of LBRanker.

In Eq. (13), ϕ is the error, n is the number of drivers in the candidate driver list, $f \in D$ is the index of the accepted driver where D is the set of all drivers in the candidate list, and x_{fj}^* is the optimal variable values. For example, if there are five drivers in the candidate driver list and the rider accepts the second driver in the ranking, then $\sum_{j \in D|j \neq f} x_{fj}^* = 3$ and $\phi = 5 - 3 - 1 = 1$. Similarly, if the rider accepts the first driver in the ranking, then $\sum_{j \in D|j \neq f} x_{fj}^* = 4$ and $\phi = 5 - 4 - 1 = 0$, meaning that the best possible scenario is realized and the resulting error is zero.

Individual errors for each trip request in case of simultaneous rank aggregation can be calculated by using Eq. (14) where ϕ_t is the error of $t \in T$, n_t is the number of drivers in the candidate list of trip request t , $f_t \in D_t$ is the index of the accepted driver where D_t is the set of all drivers in the candidate driver list of trip request t , and $x_{f_t j}^*$ is the optimal variable values of WRAMT.

$$\phi_t = n_t - \sum_{j \in D_t | j \neq f_t} x_{f_t j}^* - 1 \quad (14)$$

The NLBLP model proposed in this study aims to learn the criteria weights ω_k such that the sum of the errors ($\sum_{t \in T} \phi_t$) is minimized, since a global-based approach is inherited.

4.2. Learning the weights: Nonlinear bilevel programming model

In this section we give the details of the offline part of our ride matching procedure which learns the weights of each criteria using an NLBLP.

4.2.1. Bilevel programming

Bilevel optimization problem is an optimization problem that includes another optimization problem. The problem contains two levels of optimization tasks where main optimization task is upper-level optimization and sub optimization task is low-level that is nested in the main problem (Sinha et al., 2017). The general formulation of bilevel optimization problems is as follows:

$$\min_{x_u \in X_u, x_l \in X_l} F(x_u, x_l) \quad (15)$$

$$\text{s.t. } G_k(x_u, x_l) \leq 0 \quad \forall k = 1, \dots, K \quad (16)$$

$$x_l \in \arg\min \{f(x_u, x_l) : g_j(x_u, x_l) \leq 0, j = 1, \dots, J\} \quad (17)$$

where $F : R^n \times R^m \rightarrow R$ represents the upper level objective function, $G_k : X_u \times X_l \rightarrow R, k = 1, \dots, K$ denote the upper level constraints, $f : R^n \times R^m \rightarrow R$ represents low level objective function and $g_j : X_u \times X_l \rightarrow R, j = 1, \dots, J$ represent the lower level constraints.

The usual approaches of solving a bilevel programming problem is based on reformulation of the problem as a single level problem. In the literature, various methods are suggested to reformulate a bilevel programming problem to a single level problem such as primal Karush–Kuhn–Tucker (KKT) transformation, classical KKT transformation, and optimal value transformation (Kalashnikov et al., 2015). These methods obtain a single level model such that the optimality of lower level problem is guaranteed. The details of the reduction methods are not discussed in this paper, interested readers could refer to Kalashnikov et al. (2015).

4.2.2. Mathematical model

The mathematical formulation of the machine learning method is based on the idea of finding ω_k values for all $\pi_k \in \Pi$ such that the sum of the errors of each ranking is minimized. Let $\mathbf{1}(i \leq_k^t j)$ be an indicator function equals to 1 if driver i precedes or is in tie with driver j in criteria $k \in K$ for trip request $t \in T$ in current procedure of TAG. Then, one may found the NLBLP model below as *Rank Aggregation Training with Bilevel Programming Model (RATBLP)*. Please note that all the sets used in the formulation is defined in earlier sections with an exception. The set of criteria Π is changed with $K = \{1, 2, \dots, k, \dots, n\}$. The parameters, and decision variables are used in RATBLP is shown in Table 2:

RATBLP:

$$\min \sum_{t \in T} \phi_t \quad (18)$$

$$\text{s.t. } w_{ij}^t = 0 \quad \forall i, j \in D | i = j, \forall t \in T \quad (19)$$

$$w_{ij}^t + w_{ji}^t = 1 \quad \forall i, j \in D | i < j, \forall t \in T \quad (20)$$

$$w_{ij}^t + w_{jm}^t - w_{im}^t \geq 0 \quad \forall i, j, m \in D | i \neq j \neq m, \forall t \in T \quad (21)$$

$$w_{ij}^t - \sum_{k \in K} \mathbf{1}(i \leq_k^t j) \omega_k = 0 \quad \forall i, j \in D | i \neq j, \forall t \in T \quad (22)$$

$$\sum_{k \in K} \omega_k = 1 \quad (23)$$

Table 2
Parameters and variables of RATBLP.

Parameter	Description
n_t	Number of candidate drivers for trip request $t \in D_t$
f_t	The order of the accepted driver in trip request $t \in D_t$
Variable	Description
ϕ_t	Error of the ranking for trip request $t \in D_t$ such that $\phi_t \in \mathbb{R}$
w_{ij}^t	Superiority between driver i and driver j in trip request $t \in D_t$ such that $w_{ij}^t \in \mathbb{R}^{\oplus}$
ω_k	Weight of criteria $k \in K$ such that $\omega_k \in \mathbb{R}^{\oplus}$
x_{ij}^t	Binary variable equals to 1 if driver i precedes driver j in trip request $t \in D_t$, and 0 elsewhere

$$\phi_t - n_t - \sum_{j \in D|j=f_t} x_{ij}^t - 1 \geq 0 \quad \forall t \in T \quad (24)$$

$$w_{ij}^t \geq 0 \quad \forall i, j \in D, i \neq j, \forall t \in T \quad (25)$$

$$\min \sum_{t \in T} \sum_{i \in D} \sum_{j \in D|j>i} w_{ji}^t x_{ij}^t + w_{ij}^t x_{ji}^t \quad (26)$$

$$\text{s.t.} \quad x_{ij}^t + x_{ji}^t = 1 \quad \forall i, j \in D, i < j, \forall t \in T \quad (27)$$

$$x_{ij}^t + x_{jm}^t + x_{mi}^t \geq 1 \quad \forall i, j, m \in D, i \neq j \neq m, \forall t \in T \quad (28)$$

$$x_{ij}^t \in \{0, 1\} \quad \forall i, j \in D, i \neq j, \forall t \in T \quad (29)$$

In the above formulation, constraints (19)–(25), together with the objective function at (18), represent the upper-level problem, and Eqs. (26)–(29) represent the lower-level problem which is actually WRAMT and ensures that a valid rank aggregation is obtained for each trip request. While the entire model aims to find criteria weights such that the total error of rankings is minimized, lower-level problem ensures that each ranking is obtained through weighted rank aggregation. In the upper-level problem, constraints (19)–(21) and (25) satisfy the conditions on superiorities as discussed in earlier parts of this section. Constraints (22) calculate the superiorities by using Eq. (8). Constraint (23) ensures that sum of the weight coefficients for all criteria add up to 1, and constraints (24) calculate the errors for individual rankings for each trip request separately.

4.2.3. Solution of RATBLP : A reformulation

It is a common approach to solve bilevel programs by adding the optimality conditions of lower-level problem as constraints to upper-level problem. However, such an approach necessitates that the lower-level problem is convex (Smith and Song, 2020). Since, the lower-level problem of RATBLP is a BP, i.e., it is nonconvex, we cannot exploit such an approach. Therefore, we reformulate the model as a single-level MINLP. One may find the reformulation below as *Rank Aggregation Training with Single-level Programming Model (RATSLP)* and it is obtained by replacing the lower-level problem of RATBLP with constraints (31)–(34) and (42).

RATSLP.

$$\min \sum_{t \in T} \phi_t \quad (30)$$

$$\text{s.t.} \quad x_{ij}^t + x_{ji}^t = 1 \quad \forall i, j \in D, i < j, \forall t \in T \quad (31)$$

$$x_{ij}^t + x_{jm}^t + x_{mi}^t \geq 1 \quad \forall i, j, m \in D, i \neq j \neq m, \forall t \in T \quad (32)$$

$$w_{ji}^t x_{ij}^t + w_{ij}^t x_{ji}^t \leq w_{ij}^t \quad \forall i, j \in D, i < j, \forall t \in T \quad (33)$$

$$w_{ji}^t x_{ij}^t + w_{ij}^t x_{ji}^t \leq w_{ji}^t \quad \forall i, j \in D, i < j, \forall t \in T \quad (34)$$

$$w_{ij}^t = 0 \quad \forall i, j \in D|j = i, \forall t \in T \quad (35)$$

$$w_{ij}^t + w_{ji}^t = 1 \quad \forall i, j \in D|i < j, \forall t \in T \quad (36)$$

$$w_{ij}^t + w_{jm}^t - w_{im}^t \geq 0 \quad \forall i, j, m \in D|i \neq j \neq m, \forall t \in T \quad (37)$$

$$w_{ij}^t - \sum_{k \in K} \omega_k p_{ij}^k = 0 \quad \forall i, j \in D|i \neq j, \forall t \in T \quad (38)$$

$$\sum_{k \in K} \omega_k = 1 \quad (39)$$

$$\phi_t - n_t - \sum_{j \in D|j=f_t} x_{ij}^t - 1 \geq 0 \quad \forall t \in T \quad (40)$$

$$w_{ij}^t \geq 0 \quad \forall i, j \in D, i \neq j, \forall t \in T \quad (41)$$

$$x_{ij}^t \in \{0, 1\} \quad \forall i, j \in D, i \neq j, \forall t \in T \quad (42)$$

$$\omega_k \geq 0 \quad \forall k \in K \quad (43)$$

We present an important analytical result in the following proposition which indicates that the reformulation of RATBLP, given as RATSLP, is equivalent to RATBLP given that Assumption 1 holds, and hence allows us to solve the NLBLP as a single level MINLP:

Assumption 1. For each $i, j \in D$, $w_{ij} \neq w_{ji}$ holds.

Proposition 1. The optimum ranking of RATSLP, is also optimal for RATBLP.

Proof. By contradiction. Suppose a ranking ρ , that is not optimum for RATSLP, is optimal for RATBLP. In this ranking, there must be at least two adjacent drivers, let us say driver k precedes driver l , such that

$$w_{kl}^* < w_{lk}^* \quad (44)$$

where w_{ij}^* are optimal weights obtained at RATBLP for all $(i, j) \in D$. Eq. (44) holds thanks to the constraints (33) and (34) in RATSLP.

Perform an adjacent pairwise interchange on drivers k and l and name the new ranking ρ' . In ρ , driver k precedes driver l , while in ranking ρ' the opposite is true. All other drivers retain their actual ranks. Since k precedes l in ρ , $x_{kl} = 1$ and $x_{lk} = 0$. Also, for ranking ρ' , $x_{kl} = 0$ and $x_{lk} = 1$. Let $f(\rho)$ and $f(\rho')$ be the objective function values of RATBLP respectively for rankings ρ and ρ' . Then,

$$\begin{aligned} f(\rho) &= \sum_{i,j \in D'} w_{ij}^* x_{ji} + w_{ji}^* x_{ij} + w_{kl}^* x_{lk} + w_{lk}^* x_{kl} \\ &= \sum_{i,j \in D'} w_{ij}^* x_{ji} + w_{ji}^* x_{ij} + w_{lk}^* \\ f(\rho') &= \sum_{i,j \in D'} w_{ij}^* x_{ji} + w_{ji}^* x_{ij} + w_{kl}^* x_{lk} + w_{lk}^* x_{kl} \\ &= \sum_{i,j \in D'} w_{ij}^* x_{ji} + w_{ji}^* x_{ij} + w_{kl}^* \end{aligned}$$

where $D' = D \setminus \{(k, l)\}$.

Thus the difference of $f(\rho)$ and $f(\rho')$ is due only to driver k and l . It is easy to conclude that

$$f(\rho) > f(\rho')$$

holds. This contradicts the optimality of ρ and completes the proof of the proposition. $\square$

5. Numerical experiments and results

In this section, we present performance analysis of the proposed data-driven matching algorithm using a real-life dataset obtained from the company Lojika. The data is retrieved from the application called TAG, which is the ride pooling application of Lojika, listed in both Play Store and App Store since 2017. We work on a partial dataset, where the preferred drivers accept ride matching offers. Studied dataset contains 1338 rows, one row for each trip request; and each trip request has 14 columns: (i) candidate driver list, (ii) criteria scores for each criterion $1, 2, \dots, k, \dots, N$ where $N = 12$, and (iii) the accepted driver. The criteria used in the proposed procedure can be categorized as: criteria based on route (e.g. length of the route), criteria based on community (e.g. school/work information), criteria based on demographics (e.g. age), and criteria based on application usage (e.g. frequency of application usage). Please note that further details about the criterion set cannot be shared due to confidentiality. Details of these columns are given in the following list:

- **Candidate Driver List:** List of drivers shared in the screen of the rider e.g. $\{A, B, C\}$.
- **Criteria Scores for Criteria k :** A list of real numbers. The values are elements of a closed set $x \in \mathbb{R}$, bijective with the candidate driver list e.g. $\{A = 20, B = 10, C = 30\}$.
- **Accepted Driver:** Categorical variable indicating the accepted driver from the candidate driver list e.g. $\{B\}$.

The details of the compared solution approaches, metrics used in the comparisons and scenario analyses conducted are presented next.

The company's current approach, named "Benchmark", is considered as benchmark for the proposed solution approach. In our solution approach, one of the key components is the determination of criteria weights. These weights, ω_k in Eq. (8), can be determined by using various methods. In this study, we determine the weights by using two different methods, hence we end up with two variants of our ranking algorithm. In the first variant, we calculate the weights by using domain knowledge of Lojika experts and name this version fixed weights ranker, FWRanker. Second variant is called learning based ranker, LBRanker, which uses historical data and proposed NLBLP model to find the optimal ω_k values. The reason for proposing two variants, FWRanker and LBRanker, is to clearly show the marginal contribution of learning the criteria weights from historical data. In other words, comparing FWRanker to Benchmark reveals the marginal improvement of using rank aggregation idea through optimization; and comparing LBRanker to FWRanker shows the marginal improvement promised by learning of criteria weights using historical data.

We use Eq. (45) to calculate the weight coefficient values of FWRanker.

$$\omega'_k = \frac{\eta_k^u}{\sum_{i \in C} \eta_i^u}, \quad \forall k \in C \quad (45)$$

The equation is based on η_k^u values (from Table 1) which are determined by company experts. Once we have the weight coefficients, we construct *WRAMT* by using Eq. (8) with the calculated coefficients. Then, the candidate driver list is prepared by solving the model.

We share the details of the second approach, LBRanker, in Section 4, where we introduce a NLBLP model to learn the weight coefficients. In the training phase, *RATSLP* is used and the optimal weight coefficients ω_k^* are determined for all $k \in C$. In the testing phase, *WRAMT* is prepared for test set using these weight coefficients. We implement all models by using the GAMS IDE (Corporation, 2013); use BARON solver (Tawarmalani and Sahinidis, 2005) for *RATSLP* and CPLEX solver (Anon, 2018f) for *WRAMT*. We perform all experiments on an Intel Core i7-8500U machine with 16 GB RAM.

We compare the algorithms by using two metrics. First metric is based on the error term defined in Eq. (14). The second metric is an accuracy based performance measure, r , which is calculated as follows.

$$r = \frac{1}{|T|} \sum_{i \in T} \delta_i. \quad (46)$$

where δ_i is equal to 1 if rider's preference is ranked first in the algorithm's recommendation list for trip i , 0 otherwise. In other words, r indicates what percent of the riders' first choice is predicted correctly by the algorithm.

Before providing the results of the proposed method, we prepare the driver rankings for each trip request by using the company's methodology and calculate the errors for each trip request by following Eq. (14). We finalize the calculation by summing all errors, i.e., $\sum_{i \in T} \phi_i$. Then, we transform the criteria scores to rankings as follows: For each criterion, we sort the scores in decreasing order and replace them by their ranks. In case of tie between a set of drivers, the same rank is assigned to those drivers. For example, if the scores of each driver for a criterion are $\{A = 30, B = 30, C = 40\}$, then the ranks would be $\{A = 2, B = 2, C = 1\}$.

Table 3

Details of training and test sets.

# of drivers	Training set	Test set
2	255	110
3	195	83
4	174	74
5	174	74
6	139	60
Total	937	401

Table 4

Comparison of errors for the test set with respect to number of drivers in the time-wise split scenario.

# of drivers	Benchmark	FWRanker	LBRanker
2	0.609	0.418	0.418
3	1.205	0.855	0.880
4	2.095	0.838	0.811
5	3.243	0.757	0.581
6	2.483	2.133	0.683
Overall	1.773	0.905	0.656

We use two different scenarios to test the performances of the algorithms: (i) time-wise split, which means train and test dataset are created by considering time and (ii) time-independent split, which utilizes cross validation approach. All in all, we share the performance of ranking algorithms by using two metrics, error and accuracy, under two different split scenarios, the time-wise and the time-independent.

In the first part of analysis, the time-wise split scenario, we divide the dataset into two subsets: (i) training set and (ii) test set. The total number of trip requests used for training set is 937. This set covers data from the interval between 06/12/2018 and 08/17/2018. The test set spans fifteen-days following the ending of training dataset period. The stratified sampling method is used to create the test sets based on the length of *candidate driver list*. We consider lists of size ranging from two to six. Details of all sets are given in Table 3. First column of Table 3 gives the number of drivers in a *candidate driver list*. The following columns give the number of trip requests respectively for each length in the training and test datasets. For example, test set contains 74 trip requests with five drivers in the *candidate driver list*.

In the training phase, *RATSLP* is solved using BARON Solver (Tawarmalani and Sahinidis, 2005). It takes 665.78 s to solve the problem and the resulting error with optimal ω_k^* is found as 0.632, whereas the benchmark's error is 1.287. Note that in our context the error, ϕ , means that the driver is displayed $(\phi+1)^{\text{th}}$ position at the screen of the rider. It can be said that criteria based on route and community turn out to be more important compared to criteria based on application usage and demographics.

The performances of different ranking approaches is compared using the test dataset. Benchmark approach lists the riders' actual choice, on the average, at 2.773th position (1.773 person away from the first position, i.e., the error is 1.773), whereas FWRanker locates rider's first choice, on the average, 0.905 person away from the first position. On the other hand, the error of machine learning embedded ride matching method, LBRanker, is 0.656. This means that, LBRanker is capable of positioning rider's actual choice 1.117 and 0.249 person closer to the top recommended driver than benchmark and FWRanker, respectively.

To further investigate the performance analysis of the three ranking approaches, we choose to breakdown the error performances with respect to candidate drivers list attribute of rides. For this purpose, we categorize the candidate driver lists with respect to the number of drivers and present the error value of each approach for each case. Error value breakdown of the ranking approaches on the test set can be tracked in Table 4. First column of the table gives the number of drivers in the candidate driver list. Remaining columns show the average error values of Benchmark, FWRanker, and LBRanker approaches.

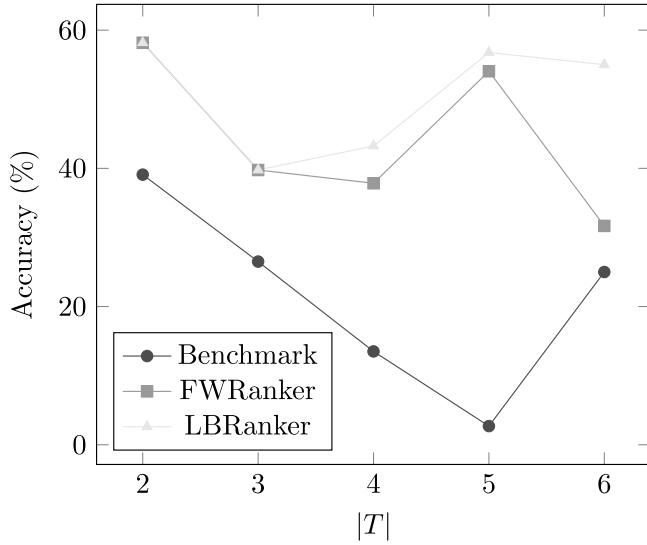


Fig. 3. Comparison of prediction accuracy breakdown for the test set with respect to number of drivers in the time-wise split scenario.

Table 5
Comparison of errors on the time-independent scenario.

	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Overall
Benchmark	1.380	1.442	1.424	1.446	1.468	1.432
FWRanker	1.271	1.273	1.249	1.251	1.199	1.249
LBRanker	0.794	1.035	0.996	1.177	1.260	1.052

The overall average errors can be seen at the last row of the table. Even though FWRanker ties with LBRanker in 2-driver case and beats LBRanker in 3-driver cases; the results reveal that LBRanker is superior to FWRanker as the number of drivers increases. LBRanker shows a significant performance for 6-driver cases, with a difference of 1.45 to its closest rival, FWRanker. We know that as the number of candidate drivers increases, the ranking problem becomes more complex due to increasing number of choices. Therefore, it can be concluded that LBRanker is more suitable than FWRanker for complex cases with larger number of candidate drivers. Both FWRanker and LBRanker dominate the Benchmark approach for all cases listed in Table 4.

We also compare ranking approaches using the second metric, accuracy. Comparison of prediction accuracy for the test set is shown in Fig. 3. Please note that, the accuracy values are calculated with respect to number of drivers in a candidate driver list. The number of drivers is presented on the x axis, where the y axis shows attained accuracy values. It is observed in Fig. 3 that prediction accuracy of LBRanker on the test set is superior to that of current methodology of the company for all number of driver cases. In comparison with FWRanker, LBRanker yields higher or at worst equal accuracy values. It is observed that, on the average, LBRanker correctly predicts the first choice of riders around 51% of the time, whereas Benchmark and FWRanker are accurate in 22% and 46% of the predictions, respectively.

In the second (i.e., the time-independent) scenario, we use a cross validation procedure to assess the robustness of the proposed approaches. We divide the data into five equal size subsets (in order to create a 5-fold setting), regardless of time information. While creating the folds, we use a stratified- k -fold sampling method that maintains the same distribution for the number of candidate drivers in every fold.

Table 5 shows that rider's actual choice is listed, on the average, at 2.432th position (1.432 person away from the first position) and 1.249th position (2.249 person away from the first position) in the recommendation list provided by Benchmark and FWRanker, respectively. On the other hand, LBRanker provides recommendation lists where the

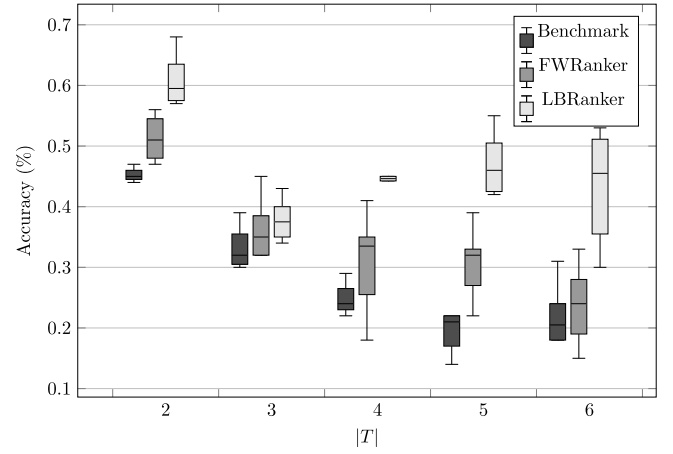


Fig. 4. Comparison of prediction accuracy breakdown with respect to number of drivers in the time-independent scenario.

actual choice of a rider is located on average 2.052th position (1.052 person away from the first position).

For the time-independent scenario, accuracy values of the approaches are also shared using box-plots, to clearly show the distribution of the performance among folds. Fig. 4 shows the accuracy distribution of each approach for each driver case. LBRanker predicts the first choice of riders in 2-driver cases with average accuracy of 61%, where average accuracy of predictions are 37%, 46%, 49%, and 45% for 3-driver, 4-driver, 5-driver, and 6-driver cases, respectively. Moreover, LBRanker's average first choice accuracy is higher than Benchmark for each fold in each driver case. Similarly, LBRanker attains higher average first choice accuracy, in all cases, when compared to FWRanker. Moreover, for 2-driver, 4-driver, and 5-driver cases LBRanker dominates FWRanker in all five-folds.

A closer inspection of Fig. 4 reveals that we obtain consistent results with earlier time-wise split scenario analyses: a similar pattern for average prediction accuracy and error performance is observed with respect to number of drivers in both scenarios. This shows that LBRanker is a valid ranking approach, promising high accuracy and low error results compared to the Benchmark approach. Moreover, comparison of FWRanker and LBRanker reveals that learning weights through computationally expensive machine learning and optimization algorithms pays off as LBRanker yields better results in almost all comparative analyses.

As a final analysis, we aggregated the results over the number of drivers and present an overall performance analysis using the weighted averages of accuracy and error values attained by all algorithms on both split scenarios. Results of the aggregated analysis is presented in Table 6. It is seen that LBRanker correctly predicts the first choice of riders 17% and 28% better compared to the benchmark for time-independent and time-wise split scenarios, respectively. Moreover, LBRanker offers recommendation lists in which the preferred driver is ranked 0.38 person closer to the rider's actual choice compared to the benchmark when time-independent split scenario is considered, and this gap widens to 1.12 person in the time-wise split scenario.

6. Conclusion

We study the ride matching problem, which has been around since the mid-20th century but gain considerably different look with the increasing availability (not only by the volume but also by variety, velocity and veracity) of data. Increasing mobile device usage, and rising popularity of sharing of under-utilized goods and services result in rise of new ecosystems. These ecosystems, with millions of potential users, made companies develop solution platforms, where users quickly

Table 6
Aggregated performance results.

Approach	Split scenario	Performance metric	
		Accuracy	Error
Benchmark	Time-wise	22.94%	1.773
FWRanker	Time-wise	45.89%	0.905
LBRanker	Time-wise	50.87%	0.656
Benchmark	Time-independent	32.17%	1.432
FWRanker	Time-independent	37.65%	1.249
LBRanker	Time-independent	48.99%	1.052

upload their needs/status and search for a provider, who is willing to satisfy needs in exchange of a moderate service fee. The version of the ride matching problem we aim to solve in this paper emanates from this transformation.

We work with a start-up company and try to develop a state-of-the-art ride matching algorithm, which utilizes historical accepted driver-rider matches to construct a recommendation engine for future trip requests. We propose a learning based matching algorithm, which consider high dimensional data while offering drivers to riders. Proposed algorithm learns importance weights of each criterion from past matchings; aggregates the rankings of each criterion (using the learned weights) and outputs a single ranking of candidate drivers for each trip request. Learning weights is done by introducing an NLBP model, which is reformulated as a single level MINLP model for the sake of reducing the computational burden. We conduct a set of real-life data based numerical analysis and show that both learning weights from historical data and using optimization to aggregate rankings significantly pay off. The proposed approach outperforms the current method used in the company by performing 17% to 28% better in predicting the first choice of riders and by ranking the preferred driver, on the average, 0.38 to 1.12 person closer to the rider's actual choice.

Although proposed algorithm clearly dominates the existing method in the company, it comes at a price. First of all, training phase requires solving a MINLP model, which may be intractable for large instances. Especially for the cases, where model needs to be trained frequently for re-calibration of weights, intractability might severely affect the overall time requirement of the algorithm. To overcome this, efficient heuristics that utilize the problem structure may be needed.

As a future research direction, we plan to implement a heuristic approach, such as math-heuristics, which combines a metaheuristic to search for weights and a mathematical model, preferably an easily solvable linear relaxation, that aggregates the rankings in order to solve large instances in reasonable amount of time. Moreover, we plan to consider implementing other learning based algorithms, such as simulation optimization, to possibly improve the performance on the prediction accuracy of the recommended matchings between generated driver rankings and rider choices.

CRediT authorship contribution statement

Ahmet Şahin: Acquisition of data, Analysis and/or interpretation of data, Writing – original draft. **İsmail Sevim:** Acquisition of data, Analysis and/or interpretation of data, Writing – original draft. **Erinç Albey:** Conception and design of study, Acquisition of data, Writing – review & editing. **Mehmet Güray Güler:** Conception and design of study, Acquisition of data, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgement

All persons who have made substantial contributions to the work reported in the manuscript (e.g., technical help, writing and editing assistance, general support), but who do not meet the criteria for authorship, are named in the Acknowledgements and have given us their written permission to be named. If we have not included an Acknowledgements, then that indicates that we have not received substantial contributions from non-authors. All authors approved the version of the manuscript to be published.

References

- Agatz, N., Erera, A., Savelsbergh, M., Wang, X., 2012. Optimization for dynamic ride-sharing: A review. *European J. Oper. Res.* 223 (2), 295–303.
- Ailon, N., Charikar, M., Newman, A., 2008. Aggregating inconsistent information: ranking and clustering. *J. ACM* 55 (5), 23.
- Aledo, J.A., Gámez, J.A., Molina, D., 2013. Tackling the rank aggregation problem with evolutionary algorithms. *Appl. Math. Comput.* 222, 632–644.
- Aledo, J.A., Gámez, J.A., Rosete, A., 2017. Partial evaluation in rank aggregation problems. *Comput. Oper. Res.* 78, 299–304.
- Anon, 2018a. Traffic's mind-boggling economic toll. <https://www.citylab.com/transportation/2018/02/traffics-mind-boggling-economic-toll/552488/>. (Accessed 25 December 2018).
- Anon, 2018b. Occupancy of cars and vans in England 2002–2017. <https://www.statista.com/statistics/314719/average-car-and-van-occupancy-in-england/>. (Accessed 25 December 2018).
- Anon, 2018c. Occupancy rates of passenger vehicles. <https://www.eea.europa.eu/data-and-maps/indicators/occupancy-rates-of-passenger-vehicles/occupancy-rates-of-passenger-vehicles/>. (Accessed 25 December 2018).
- Anon, 2018d. Traffic's mind-boggling economic toll. <https://www.citylab.com/transportation/2018/02/traffics-mind-boggling-economic-toll/552488/>. (Accessed 25 December 2018).
- Anon, 2018e. Drivers around the world lose an average of 8 working days per year due to traffic congestion. <https://corporate.tomtom.com/news-releases/news-release-details/drivers-around-world-lose-average-8-working-days-year-due?ReleaseID=804498>. (Accessed 25 December 2018).
- Anon, 2018f. IBM CPLEX optimizer. <https://www.ibm.com/analytics/cplex-optimizer>. (Accessed 25 December 2018).
- Anon, 2020. INRIX global traffic scorecard. <http://inrix.com/scorecard/>. (Accessed 05 October 2020).
- Badal, P.S., Das, A., 2018. Efficient algorithms using subiterative convergence for Kemeny ranking problem. *Comput. Oper. Res.* 98, 198–210.
- Baskin, J.P., Krishnamurthi, S., 2009. Preference aggregation in group recommender systems for committee decision-making. In: *Proceedings of the Third ACM Conference on Recommender Systems*. ACM, pp. 337–340.
- Betzler, N., Brederbeck, R., Niedermeier, R., 2014. Theoretical and empirical evaluation of data reduction for exact Kemeny rank aggregation. *Auton. Agents Multi-Agent Syst.* 28 (5), 721–748.
- Bozdog, N.V., Makkes, M.X., Van Halteren, A., Bal, H., 2018. Ridematcher: peer-to-peer matching of passengers for efficient ridesharing. In: *Proceedings of the 18th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing*. IEEE Press, pp. 263–272.
- Brin, S., Page, L., 1998. The anatomy of a large-scale hypertextual web search engine. *Comput. Netw. ISDN Syst.* 30 (1–7), 107–117.
- Conitzer, V., Davenport, A., Kalagnanam, J., 2006. Improved bounds for computing Kemeny rankings. In: *AAAI*, Vol. 6, pp. 620–626.
- Coppersmith, D., Fleischer, L.K., Rurda, A., 2010. Ordering by weighted number of wins gives a good ranking for weighted tournaments. *ACM Trans. Algorithms (TALG)* 6 (3), 55.
- Cordeau, J.-F., Laporte, G., 2007. The dial-a-ride problem: models and algorithms. *Ann. Oper. Res.* 153 (1), 29–46.
- Corporation, G.D., 2013. General algebraic modeling system (GAMS) release 24.2.1. URL <http://www.gams.com/>.
- Cramer, J., Krueger, A.B., 2016. Disruptive change in the taxi business: The case of uber. *Amer. Econ. Rev.* 106 (5), 177–182.
- Dwork, C., Kumar, R., Naor, M., Sivakumar, D., 2001. Rank aggregation methods for the web. In: *Proceedings of the 10th International Conference on World Wide Web*. ACM, pp. 613–622.
- Fan, J., Xu, J., Hou, C., Cao, B., Dong, T., Cheng, S., 2018. URoad: An efficient algorithm for large-scale dynamic ridesharing service. In: *2018 IEEE International Conference on Web Services. ICWS*, IEEE, pp. 9–16.
- Furuhata, M., Dessouky, M., Ordóñez, F., Brunet, M.-E., Wang, X., Koenig, S., 2013. Ridesharing: The state-of-the-art and future directions. *Transp. Res. B* 57, 28–46.
- Herbawi, W., Weber, M., 2012. The ridematching problem with time windows in dynamic ridesharing: A model and a genetic algorithm. In: *2012 IEEE Congress on Evolutionary Computation*. IEEE, pp. 1–8.

- Hou, L., Li, D., Zhang, D., 2018. Ride-matching and routing optimisation: Models and a large neighbourhood search heuristic. *Transp. Res. E* 118, 143–162.
- Jacob, J., Roet-Green, R., 2018. Ride solo or pool: Designing price-service menus for a ride-sharing platform, Working Paper No: 3008136, University of Rochester, Simon Business School.
- Kalashnikov, V.V., Dempe, S., Pérez-Valdés, G.A., Kalashnykova, N.I., Camacho-Vallejo, J.-F., 2015. Bilevel programming and applications. *Math. Probl. Eng.* 2015.
- Kumar, P., Khani, A., 2021. An algorithm for integrating peer-to-peer ridesharing and schedule-based transit system for first mile/last mile access. *Transp. Res. C* 122, 102891.
- Li, B., Krushinsky, D., Reijers, H.A., Van Woensel, T., 2014. The share-a-ride problem: People and parcels sharing taxis. *European J. Oper. Res.* 238 (1), 31–40.
- Long, J., Tan, W., Szeto, W., Li, Y., 2018. Ride-sharing with travel time uncertainty. *Transp. Res. B* 118, 143–171.
- Masmoudi, M.A., Braekers, K., Masmoudi, M., Dammak, A., 2017. A hybrid genetic algorithm for the heterogeneous dial-a-ride problem. *Comput. Oper. Res.* 81, 1–13.
- McKenzie, B., 2015. Who Drives To Work?: Commuting By Automobile in the United States: 2013. US Department of Commerce, Economics and Statistics Administration, US
- Nourinejad, M., Roorda, M.J., 2016. Agent based model for dynamic ridesharing. *Transp. Res. C* 64, 117–132.
- Pihur, V., Datta, S., Datta, S., 2007. Weighted rank aggregation of cluster validation measures: a monte carlo cross-entropy approach. *Bioinformatics* 23 (13), 1607–1615.
- Riedler, M., Raidl, G., 2018. Solving a selective dial-a-ride problem with logic-based benders decomposition. *Comput. Oper. Res.* 96, 30–54.
- Saaty, T.L., 1988. What is the analytic hierarchy process? In: *Mathematical Models for Decision Support*. Springer, pp. 109–121.
- Sánchez, D., Martínez, S., Domingo-Ferrer, J., 2016. Co-utile P2P ridesharing via decentralization and reputation management. *Transp. Res. C* 73, 147–166.
- Schalekamp, F., Zuylen, A.v., 2009. Rank aggregation: Together we're strong. In: 2009 Proceedings of the Eleventh Workshop on Algorithm Engineering and Experiments. ALENEX, SIAM, pp. 38–51.
- Siddiqi, Z., Buliung, R., 2013. Dynamic ridesharing and information and communications technology: past, present and future prospects. *Transp. Plan. Technol.* 36 (6), 479–498.
- Sinha, A., Malo, P., Deb, K., 2017. A review on bilevel optimization: From classical to evolutionary approaches and applications. *IEEE Trans. Evol. Comput.* 22 (2), 276–295.
- Smith, J.C., Song, Y., 2020. A survey of network interdiction models and algorithms. *European J. Oper. Res.* 283 (3), 797–811.
- Ta, N., Li, G., Zhao, T., Feng, J., Ma, H., Gong, Z., 2017. An efficient ride-sharing framework for maximizing shared route. *IEEE Trans. Knowl. Data Eng.* 30 (2), 219–233.
- Tafreshian, A., Masoud, N., Yin, Y., 2020. Frontiers in service science: Ride matching for peer-to-peer ride sharing: A review and future directions. *Serv. Sci.* 12 (2–3), 44–60.
- Tawarmalani, M., Sahinidis, N.V., 2005. A polyhedral branch-and-cut approach to global optimization. *Math. Program.* 103, 225–249.
- Thangaraj, R.S., Mukherjee, K., Raravi, G., Metrewar, A., Annamaneni, N., Chattopadhyay, K., 2017. Xhare-a-ride: A search optimized dynamic ride sharing system with approximation guarantee. In: 2017 IEEE 33rd International Conference on Data Engineering. ICDE, IEEE, pp. 1117–1128.
- Tian, C., Huang, Y., Liu, Z., Bastani, F., Jin, R., 2013. Noah: a dynamic ridesharing system. In: Proceedings of the 2013 ACM SIGMOD International Conference on Management of Data. ACM, pp. 985–988.